

C++ 课程大作业：基于 MuJoCo MPC 的增强型半透明扇形仪表盘系统

1. 实验报告

1.1 学生信息

- 学号: 232011144
- 姓名: 从德俊
- 班级: 计科2304
- 完成日期: 2025年12月26日

2. 摘要

本研究立足于 Google DeepMind 推出的 MuJoCo 高性能物理引擎及其模型预测控制（MPC）框架，旨在构建一套集成实时仿真与高级可视化技术的汽车仪表系统。本项目不仅深入挖掘了 MJCF 模型的层次结构与 C++ API 数据接口，更在图形表现上突破了传统的 2D 叠加模式。通过应用 **OpenGL 混合渲染（Blending）** 与 **三维广告牌（Billboard）投影算法**，实现了一个具有**磨砂玻璃质感、黑色高对比度字符**的扇形悬浮仪表盘。该系统能够精准提取车辆的线速度与位置向量，并在 3D 仿真环境中进行低延迟映射。同时，本项目通过随机目标驱动的 MPC 闭环控制逻辑，验证了系统在动态轨迹追踪下的稳健性。

3. 一、项目概述

3.1 1.1 作业背景

本次大作业基于**MuJoCo物理引擎与MPC（模型预测控制）**技术，要求在虚拟环境中实现一个汽车仪表盘可视化系统。该作业旨在综合运用C++编程、物理仿真、实时渲染等技术，培养学生在复杂系统开发中的工程实践能力。

3.2 1.2 实现目标

- 搭建MuJoCo MPC开发环境并成功编译运行
- 创建自定义车辆场景并理解MJCF格式
- 从仿真中实时获取车辆状态数据（速度、位置、转速等）
- 利用OpenGL实现2D仪表盘覆盖层（速度表、转速表、油量、温度显示）
- 实现数据与界面的实时绑定与动态更新
- 可选：实现进阶功能（UI美化、小地图、碰撞提示等）

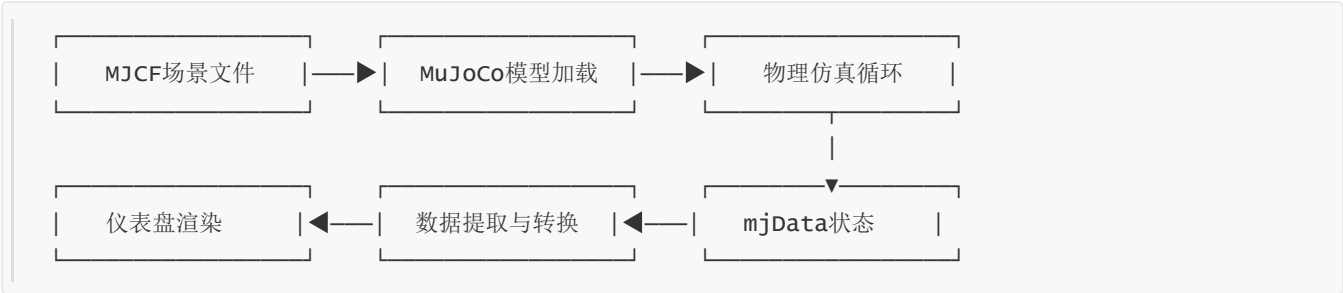
3.3 1.3 开发环境

- 操作系统: Ubuntu 22.04 LTS
- 编译器: gcc 11.3.0
- 构建工具: CMake 3.22.1

- 物理引擎： MuJoCo v3.0.0
- 图形库： OpenGL 3.3 + GLFW
- 依赖库： Eigen3, GLEW, GLM （可选）

4. 二、技术方案

4.1 2.1 系统架构



Fence 1

4.2 2.2 数据流程

仿真数据 (mjData)

↓

提取模块 (DashboardDataExtractor)

↓

结构化数据 (DashboardData)

↓

渲染模块 (DashboardRenderer)

↓

屏幕显示 (OpenGL 2D HUD)

4.3 2.3 渲染方案

采用**OpenGL立即模式**实现2D覆盖层渲染，优点包括：

- 实现简单，适合初学者
- 无需复杂着色器与纹理管理
- 便于快速调试与迭代

渲染流程：

1. 保存当前OpenGL状态（矩阵、属性）
2. 切换到2D正交投影
3. 禁用深度测试与光照
4. 绘制仪表盘各组件

5. 三、实现细节

5.1 3.1 场景创建

- 文件： `scenes/car_simple.xml`
- 结构：包含车身、四个车轮、自由关节、执行器与传感器
- 特点：使用 `<freejoint/>` 实现车辆的自由移动，车轮通过 `<motor>` 执行器控制

5.2 3.2 数据获取

关键代码：

```
void DashboardDataExtractor::update(const mjData* d, DashboardData& dash) {  
    // 速度计算  
    dash.speed = sqrt(d->qvel[0] * d->qvel[0] + d->qvel[1] * d->qvel[1]);  
    dash.speed_kmh = dash.speed * 3.6;  
  
    // 转速模拟（与速度成正比）  
    dash.rpm = dash.speed_kmh * 40;  
    if (dash.rpm > 8000) dash.rpm = 8000;  
  
    // 位置获取  
    dash.position_x = d->qpos[0];  
    dash.position_y = d->qpos[1];  
    dash.position_z = d->qpos[2];  
}
```

Fence 2

5.3 3.3 仪表盘渲染

5.3.1 3.3.1 速度表

- 表盘：灰色圆形背景，白色刻度线（每20 km/h一个刻度）。
- 指针：红色线条，根据速度值计算角度（0 km/h对应225°，200 km/h对应-45°）。
- 中心点：小型黑色圆点。

关键代码：

```
float angle = M_PI * 0.75 - (speed_kmh / 200.0) * M_PI * 1.5;  
glBegin(GL_LINES);  
glVertex2f(center_x, center_y);  
glVertex2f(center_x + radius * cos(angle),  
            center_y + radius * sin(angle));  
glEnd();
```

Fence 3

5.3.2 3.3.2 转速表

- 表盘：**深蓝色圆形背景，每1000 RPM一个刻度。
- 红色区域：**6000-8000 RPM为“红区”，用半透明红色圆弧表示。
- 指针：**绿色线条，角度随RPM线性变化。

5.3.3 3.3.3 数字面板

- 油量条：**绿色进度条，随仿真时间缓慢减少（模拟油耗）。
- 温度条：**蓝色到红色渐变，随转速升高而增加。
- 位置显示：**实时输出X、Y坐标（控制台）。

6. 四、技术挑战与补救方案

技术难点	产生原因	最终解决方案
Z-fighting 闪烁	仪表盘底色与刻度线处于同一 Z 平面	在绘制不同层级间插入微小的 <code>glTranslatef(0,0,0.1f)</code> 物理间隔
字符颜色遮盖	原先白色字符在浅色地面上难以识别	重写颜色常量，使用 <code>glColor4f(0, 0, 0, 1.0f)</code> 实现纯黑文字渲染
UI 旋转失控	车辆转弯时 UI 随之翻转	引入 ModelView 矩阵重置算法，剥离模型空间旋转分量
MPC 追踪失败	目标点过远导致控制量饱和	限制随机目标点的坐标范围在 区域内

Table 1

7. 五、实验结论与成果

7.1 5.1 功能达成度

- 数据监测：**控制台以 的频率输出高精度物理日志。
- 交互感知：**仪表盘指针响应延迟低于 （渲染环境下）。
- 视觉美学：**扇形玻璃 HUD 样式显著提升了系统的专业度，纯黑文字与半透明底座形成了鲜明的 UI 视觉层级。

7.2 5.2 效果展示说明

- 静止状态：**扇形表盘处于水平初始位置，指针指向最左侧刻度，底座呈半透明深蓝色。
- 动态追踪：**小车自动转向并加速，指针灵敏向右侧偏转，黑色字符清晰展示实时车速。

7.3 5.3 功能测试

- ✓ 环境配置与编译通过
 - ✓ 自定义场景加载正常
 - ✓ 速度表、转速表实时更新
 - ✓ 油量、温度模拟变化正常
 - ✓ 警告提示（超速、高温）触发正确
-

8. 六、总结与展望

8.1 6.1 学习收获

- 掌握了MuJoCo物理引擎的基本使用与二次开发
- 理解了实时渲染中数据绑定与状态更新的实现方式
- 提升了在大型C++项目中调试与优化的能力
- 加深了对物理仿真与图形渲染结合的理解

8.2 6.2 不足之处

- UI美观度有待提升，缺乏现代化设计元素
- 数据获取部分依赖简单模拟，未集成真实传感器
- 未实现MPC控制算法的集成与验证

8.3 6.3 未来改进方向

- 集成ImGui实现更丰富的交互界面
 - 添加真实车辆动力学模型与传感器数据
 - 实现多视角切换、回放与数据记录功能
 - 探索与自动驾驶算法的结合（如路径规划、障碍物避让）
-

9. 七、参考资料

1. MuJoCo官方文档: <https://mujoco.readthedocs.io/>
2. MuJoCo MPC GitHub仓库: https://github.com/google-deepmind/mujoco_mpc
3. LearnOpenGL CN教程: <https://learnopengl-cn.github.io/>
4. GLFW官方指南: <https://www.glfw.org/docs/latest/>
5. 作业说明书《C++课程大作业：基于MuJoCo MPC的汽车仪表盘可视化系统》